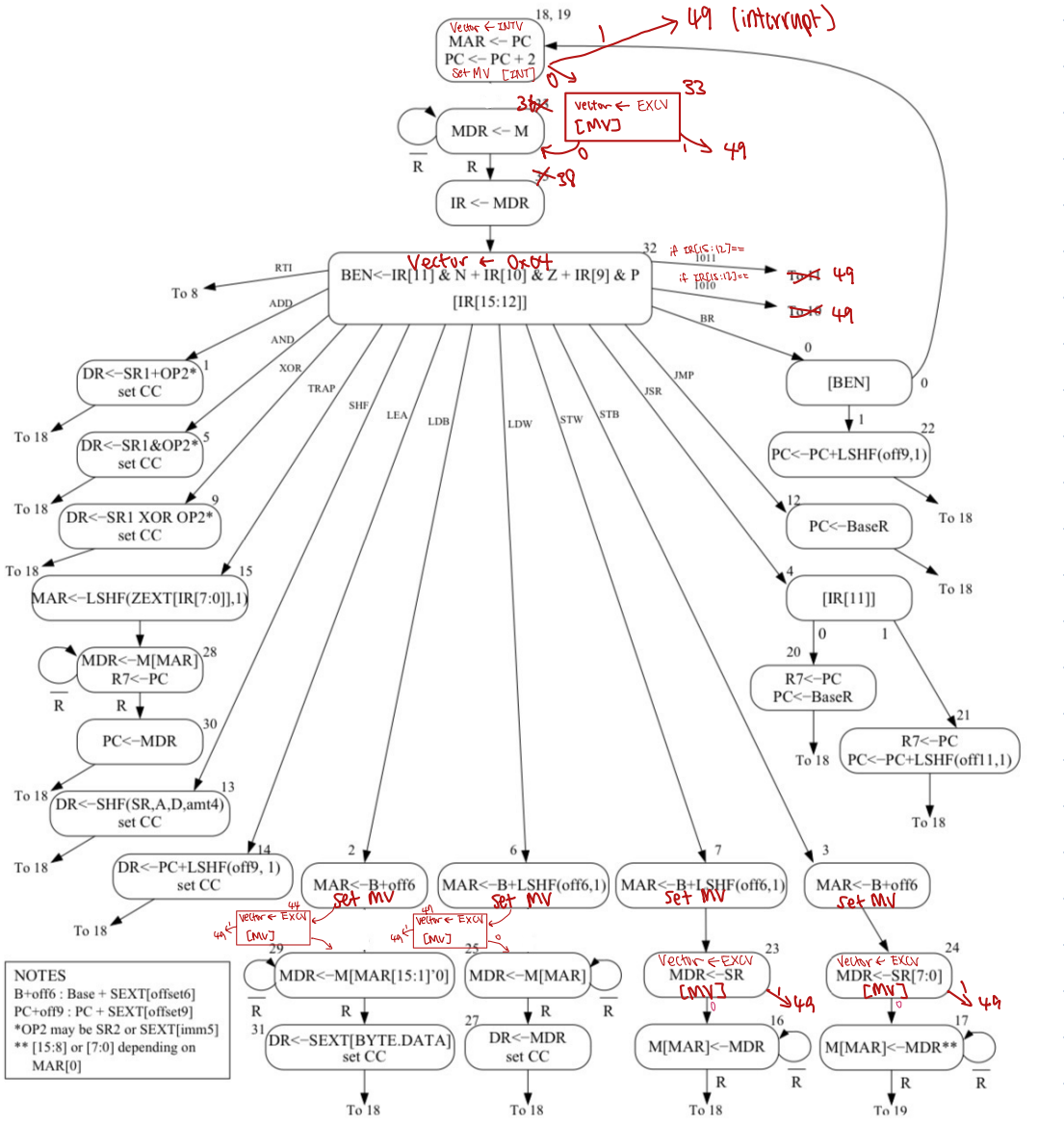
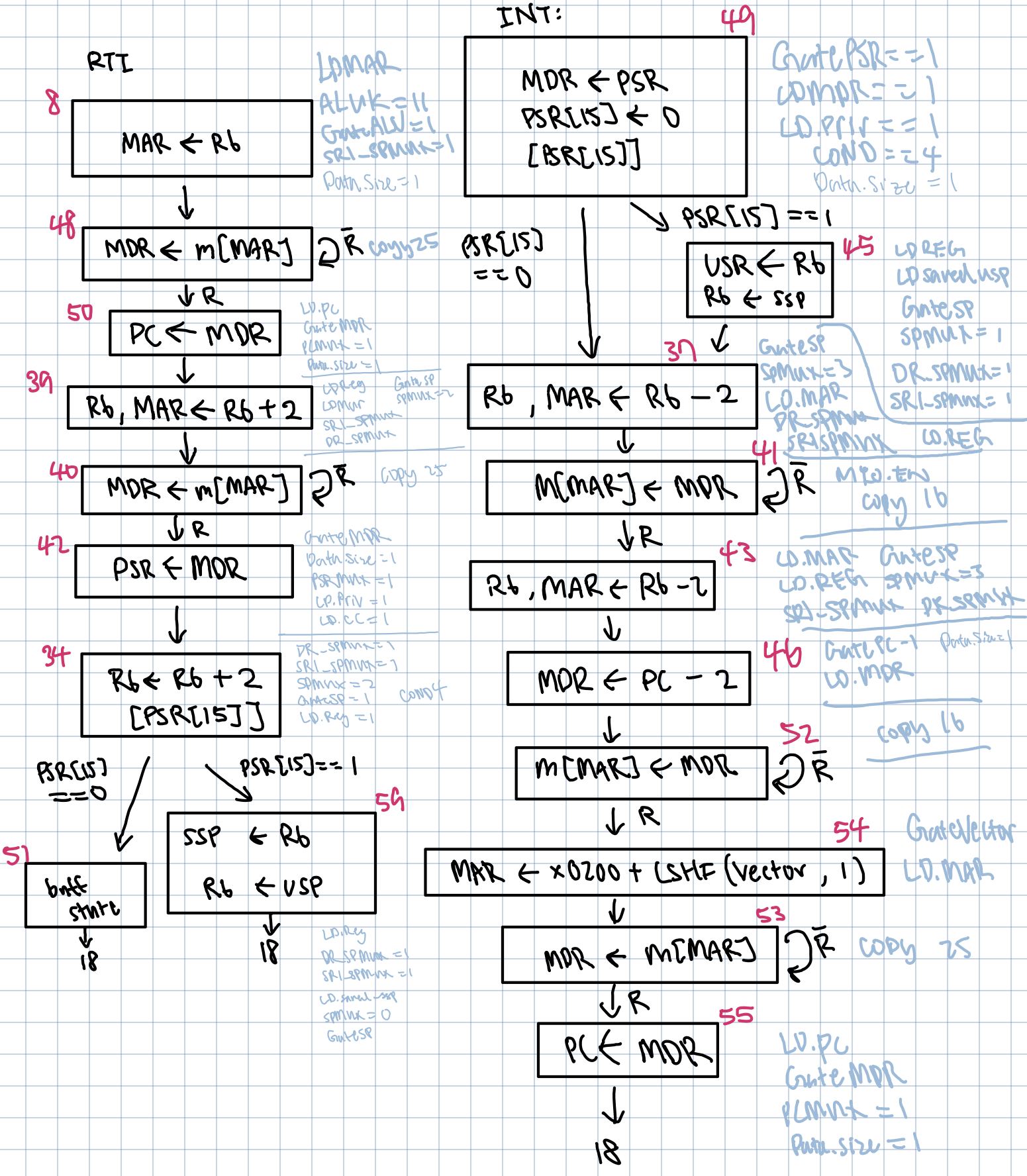


Figure C.2: A state machine for the LC-3b

New states or changes to existing states (in red above, and black boxes in next page):

- 18, 19 - put INTV in Vector register, set the MV (memory violation) register by checking protection or unaligned access exceptions with logic block. Also checked for interrupt bit to see if there's any interrupt waiting to be triggered & processed.
- 2, 3, 6, 7 - added set MV like state 18 and 19
- 33, 44, 47 - load Vector register with exception vector, which depends on the output of VectorMUX, which depends on further logic to differentiate between protection and unaligned access exceptions. Also checks for if MV register is a 1, because if it is, then there's a memory violation exception (protection or unaligned access), and it goes to state 49, or continues to the next normal state otherwise.
- 23, 24 - same as 33, 44, 47, but embedded along with existing states


RTI
8
MAR ← R6
LDMAR
ALUK = 11
GateALU = 1
SR1_SPMUX = 1
Data.Size = 1
48
MDR ← m[MAR]
R
copy 25
R
50
PC ← MDR
LD.PC
GateMDR
PCMUX = 1
Data.size = 1
39
R6, MAR ← R6 + 2
LDReg
LDMar
SR1_SPMUX
DR_SPMUX
GateSP
SPMUX = 2
40
MDR ← m[MAR]
R
copy 25
42
PSR ← MDR
GateMDR
Data.Size = 1
PSRMUX = 1
LD.Priv = 1
LD.CC = 1
34
R6 ← R6 + 2
[PSR[15]]
DR_SPMUX = 1
SR1_SPMUX = 1
SPMUX = 2
GateSP = 1
LD.Reg = 1
COND4
PSR[15] == 0
PSR[15] == 1
57
both start
18
54
SSP ← R6
R6 ← USP
18
LD.Reg
DR_SPMUX = 1
SR1_SPMUX = 1
LD.saved_ssp
SPMUX = 0
GateSP
INT:
49
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
GatePSR = 1
LDMDR = 1
LD.Priv = 1
COND = 4
Data.Size = 1
PSR[15] == 1
PSR[15] == 0
45
USR ← R6
R6 ← SSP
LD.REG
LD.savedUSP
GateSP
SPMUX = 1
37
R6, MAR ← R6 − 2
GateSP
SPMUX = 3
LD.MAR
DR_SPMUX
SR1SPMUX
DR_SPMUX = 1
SR1_SPMUX = 1
LD.REG
41
M[MAR] ← MDR
R
MIO.EN
copy 16
43
R6, MAR ← R6 − 2
LD.MAR GateSP
LD.REG SPMUX = 3
SR1_SPMUX DR_SPMUX
46
MDR ← PC − 2
GatePC = 1 Data.Size = 1
LD.MDR
copy 16
52
m[MAR] ← MDR
R
54
MAR ← x0200 + LSHF(vector, 1)
GateVector
LD.MAR
53
MDR ← m[MAR]
R
copy 25
55
PC ← MDR
LD.PC
GateMDR
PCMUX = 1
Data.size = 1
18

New control signals

- GatePC-1 - to preserve architectural states when saving the state of the machine before exceptions/interrupts
- LD.Saved_SSP, LD.Saved_USP, SPMUX, GateSP. DR_SPMUX, SR1_SPMUX - for storing, fetching, incrementing, decrementing, and selecting the stack pointer for user and supervisor stacks
- PSRMUX, GatePSR, LD.Priv - for new PSR functionalities and PSR storage (like LD.Priv)
- LD.MV - for loading into the memory violation reg
- CHK.MV, CHK.IOP, COND3 - for controlling new conditional state branching
- GateVector, LD.Vector, VectorMUX - for interrupt & exception vectors

More new states for RTI and exception/interrupt handling

- 49, 45, 37, 41, 43, 46, 52, 54, 53, 55 - conditionally saves the user stack pointer and fetches the supervisor stack pointer, pushes PSR, PC into the supervisor stack while decrementing the stack pointer, and finally loads the address of the ISR into the PC by first accessing the interrupt/exception vector table.
- 8, 48, 50, 39, 40, 42, 34, 59, 51 - pops PC, PSR off the supervisor stack while incrementing the stack pointer, then conditionally saves the supervisor stack pointer and fetches the user stack pointer depending on the privilege of the program that the RTI is returning to.

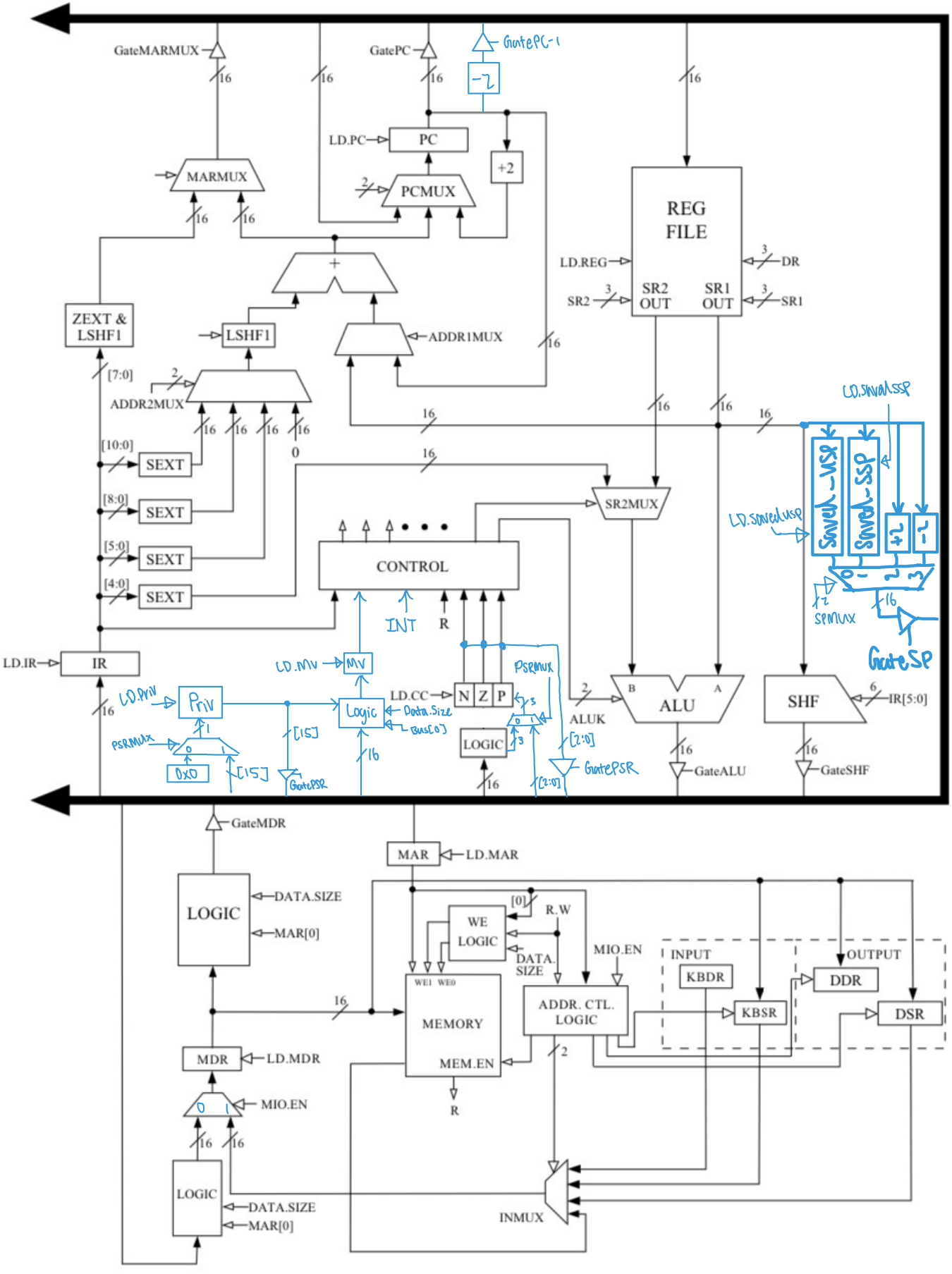
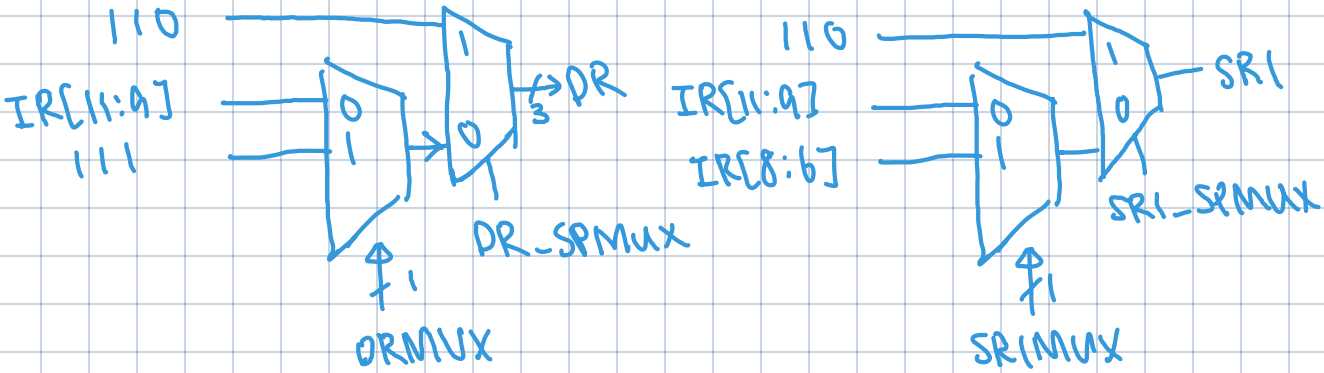
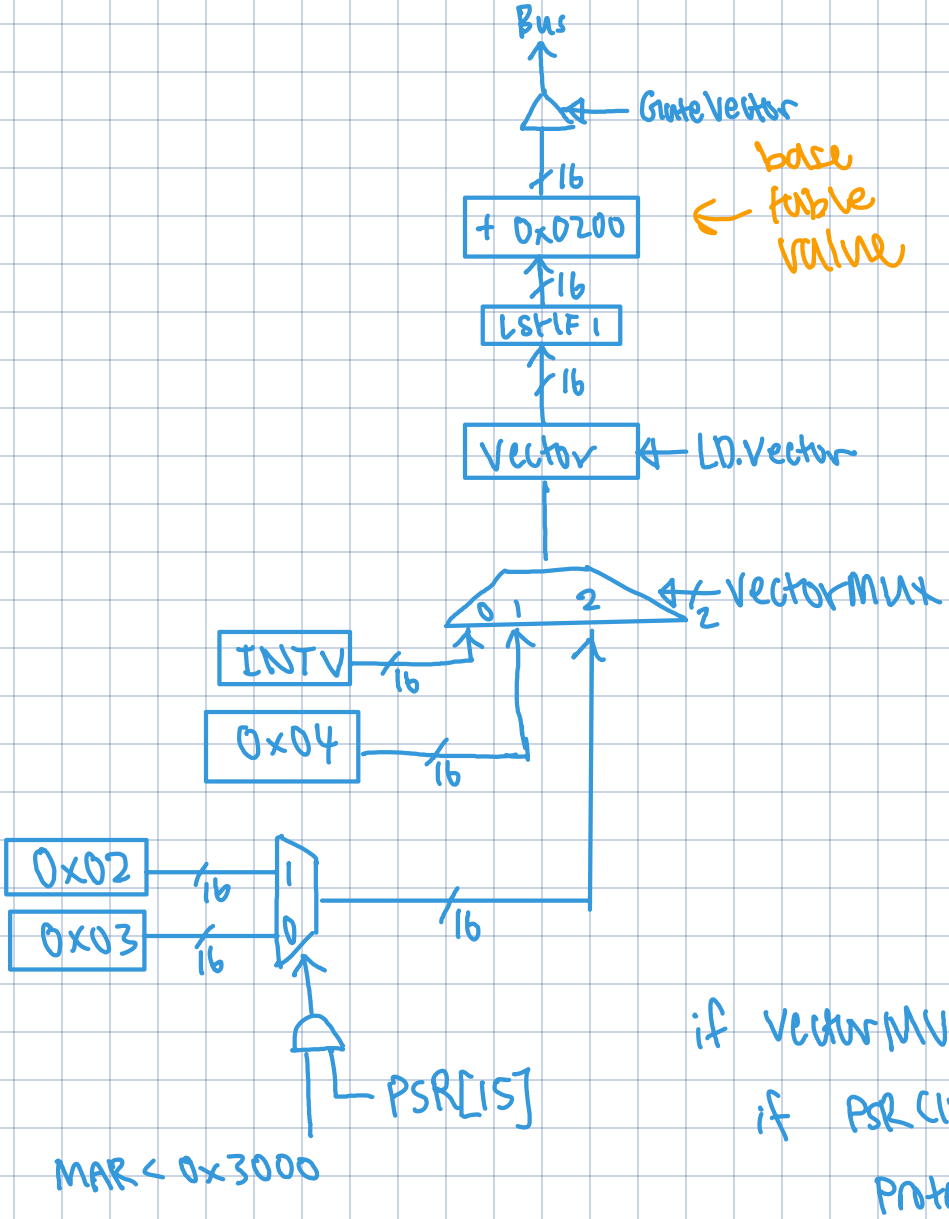


Figure C.3: The LC-3b data path





only when testing for MV

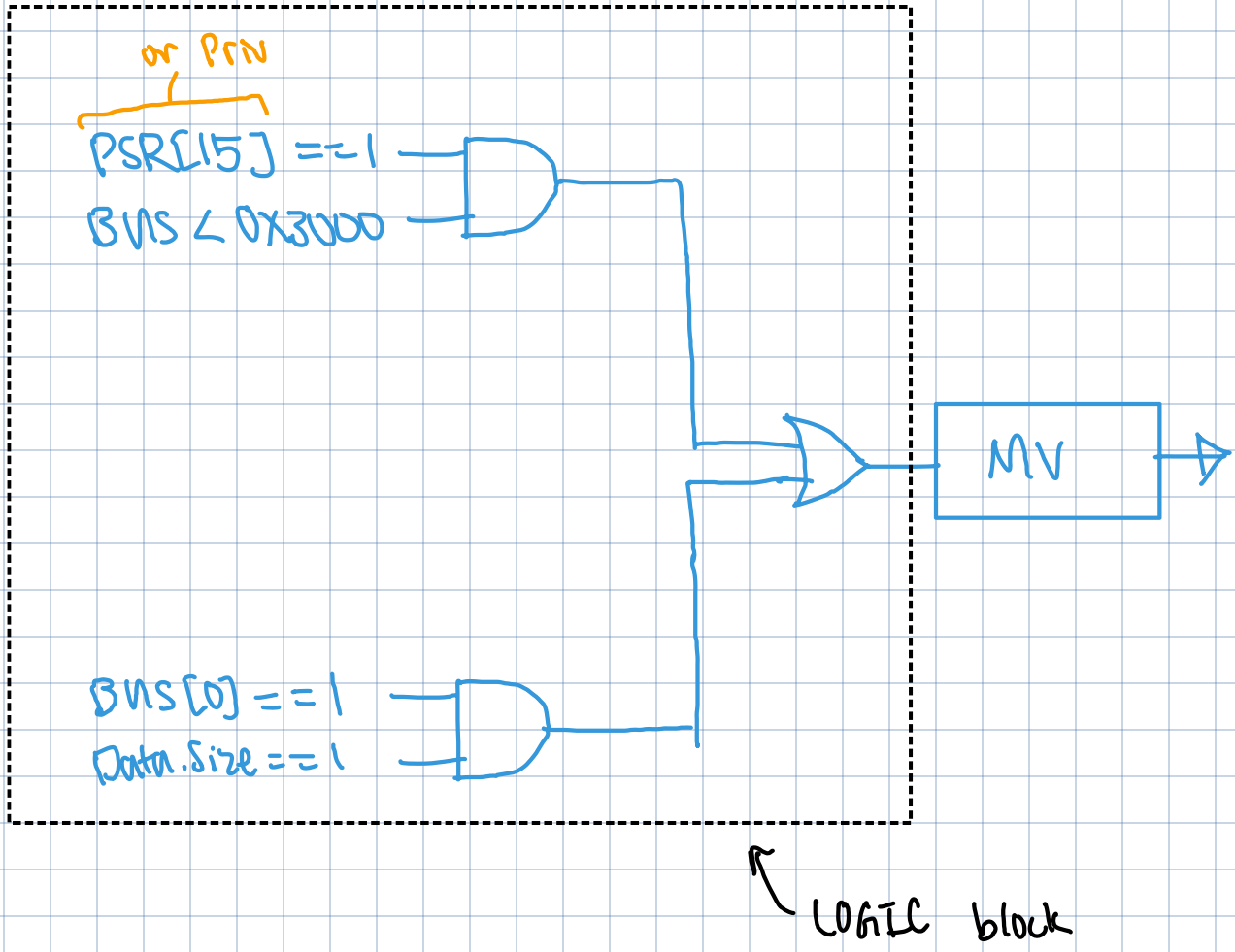
if vectorMUX == 2

if PSR[15] == 1 AND MAR < 0x3000

protection exception

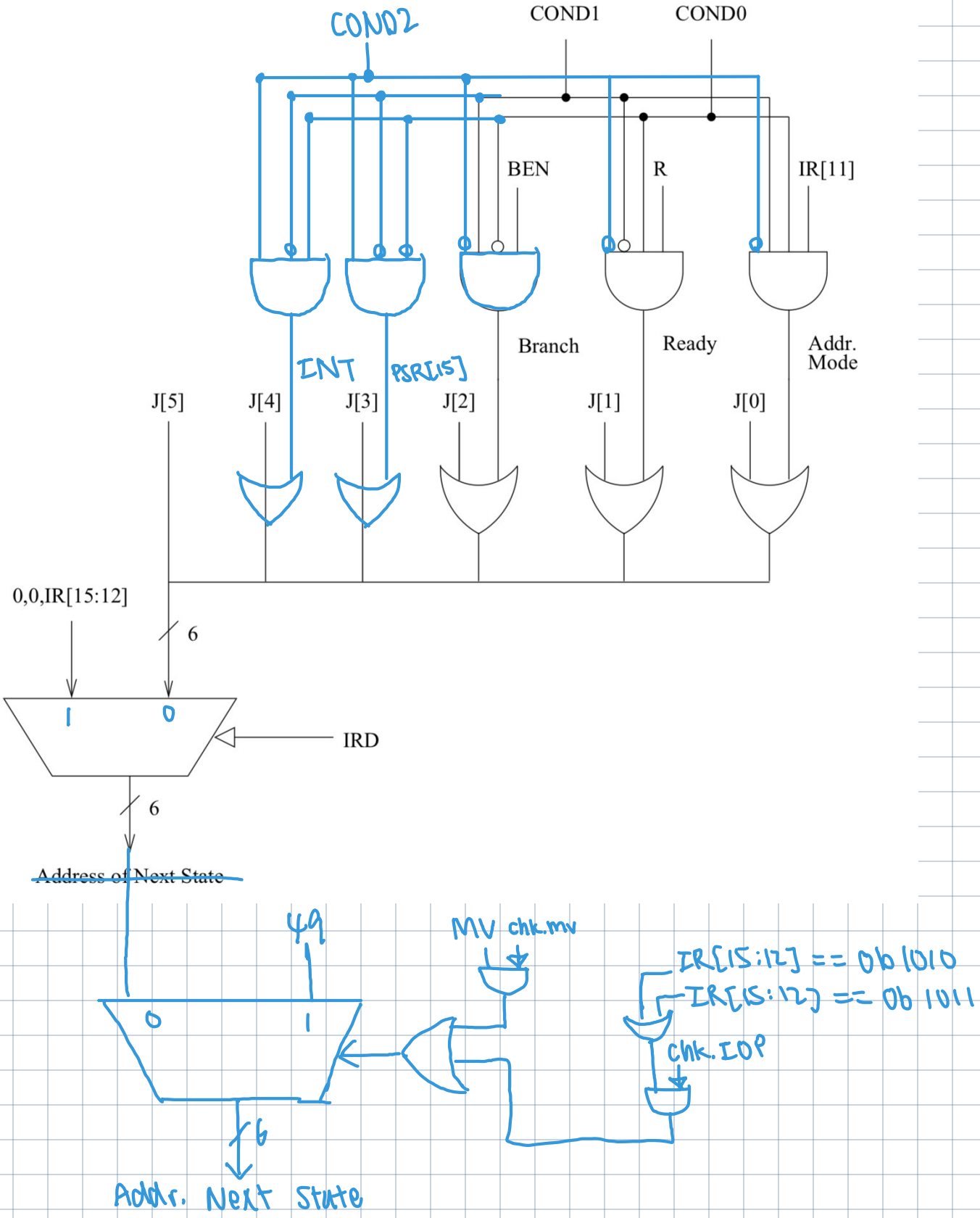
else ~~if MAR[0] == 1 AND DataSize == 1~~

unaligned access



Data Path Explanations

- PSR structures: includes one bit register called Priv and some logic to load and store between NZP bits and bus.
- MV register & logic: helps determine if there's a memory violation (protection or unaligned access exceptions)
- Stack pointers structures (more DR and SR1 MUXs and Saved_USP, Saved_SSP registers, with -2 and +2 incrementing logic. This helps with the pushing and popping of PC and PSR on the stack when context switching for interrupts & exceptions and RTI.
- GatePC-1 helps put a decremented PC on the bus. This helps with exception handling and interrupt handling to preserve the machine's architectural registers in its original states (to ensure that exception causing instructions don't change the machine's states). This decremented pc is pushed onto the stack during context switches.
- I also added exception and interrupt vector register to store the vector address to be used later. This helps with saving the number of states by inserting addresses in this register along with other earlier states (earlier than interrupt/exception handling).


COND2
COND1
COND0
BEN
R
IR[11]
INT
PSR[15]
Branch
Ready
Addr. Mode
J[5]
J[4]
J[3]
J[2]
J[1]
J[0]
0,0,IR[15:12]
6
1
0
IRD
6
Address of Next State
49
MV chk.mv
IR[15:12] == 0b1010
IR[15:12] == 0b1011
chk.IOP
0
1
6
Addr. Next State

Microsequencer changes

- added one more COND bit in order to handle more conditional state branching. This is needed in order to support the INT conditional transition and the privilege conditional transition (for saving/fetching stack pointers)
- Added one more MUX for exception handling. This forces certain states to transition state 49 (start of context switching) if there is an exception. If not, then it continues with normal operation with the J-fields. This helps save states because it's no longer limited by the j-field. It can branch to a state (state 49) that is not specified in the j-field

Signal Name	Signal Values	
J/6:		
COND/3:	$COND_0$	;Unconditional
	$COND_1$	;Memory Ready
	$COND_2$	;Branch
	$COND_3$	;Addressing Mode
IRD/1:	NO, YES	

CHK.MV / 1
CHK.IOP/1

$COND_4$: PSR[15]
$COND_5$: INT

Table C.2: Microsequencer control signals